

Architecture and Trade-offs

How the in-memory ledger behaves today, where it reaches its limits, and the smallest useful steps toward production.

1. Append-only at 100x volume

The program processes events in arrival order. Accepted financial changes become new ledger entries; existing entries are never edited. Balances use the entry's value day, while authorization holds are kept separately because they reduce available funds without changing the ledger balance.

At 100x volume, repeated calculation becomes a problem before storage does.

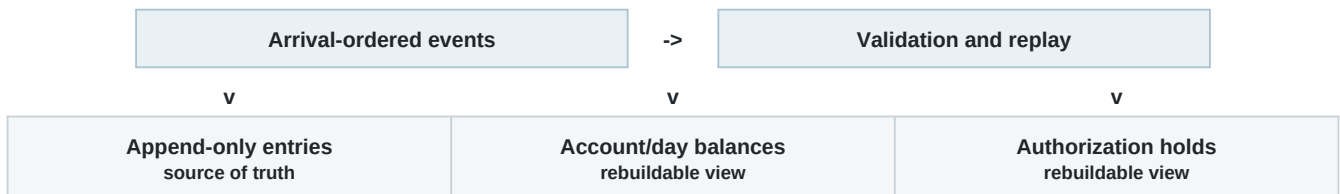
What breaks first

Operation	What happens as history grows
Balance	Reads every ledger entry for each answer.
Fees and interest	Repeat balance scans across several days.
Reversal	Copies and searches the full entry list.
Available balance	Scans all authorization records for active holds.

The growing state is not only the ledger. Authorization records, fee markers, reversal markers, errors, interest records, and the range of historical days also grow. The current six-day example hides that long-term cost.

Cheapest structural change

Add one rebuildable view per account. This view stores daily balance changes, the active hold total, and direct references to source events. A late entry then rebuilds only that account from the affected day, instead of searching the whole ledger.



Trade-off. The view makes reads faster but creates a second representation that could become inconsistent. It must be rebuildable from the append-only entries and checked regularly. Durable storage, partitioning, snapshots, and archival remain later steps; they are not needed to solve the first 100x bottleneck.

2. Value dating and authorization lifecycle

Value-dated entries in production

A value-dated entry is received today but belongs financially to an earlier day. For a UAE-licensed bank, changing that earlier balance may also change fees, interest, customer statements, reconciliation results, and regulatory reports.

Keep these dates separate	Why
Transaction time	When the real-world activity happened.
Value date	Which day's financial balance it changes.
Recorded time	When the bank received and posted it.

The original record must remain visible. Silently rewriting it would hide what the bank knew at the time and make statements, investigations, and reconciliation harder to explain.

One control before launch: controlled backdating

1	Detect a value date earlier than the recorded date.
2	Show which balances, fees, interest, statements, and reports change.
3	Require approval by a second authorized employee.
4	Record the reason, evidence, dates, and both employees.
5	Post a new entry, then rerun and reconcile affected results.

This control follows the direction of the cited CBUAE standards: consumers must receive detailed transaction information, while bank controls should include clear approvals, separation of duties, dual control, and reconciliation.^{[1][2]}

Authorization lifecycle

In the implemented model, rejection is the only final outcome other than settlement. An invalid settlement attempt does not end an existing authorization.

Situation	Current behavior
Not enough available funds	Reject the request; create no hold.
Known active reference; account and currency match	Post the actual debit, release the full hold, mark settled.
Unknown or no longer active reference	Reject the settlement; change no balance.
Wrong account or currency	Reject the settlement; keep the existing hold active.
No settlement arrives	Keep the hold active indefinitely.

Required production additions

- **Expiry:** release an unused hold after a defined time.
- **Cancellation:** release the hold without creating a debit.
- **Partial settlement:** post the actual debit and apply a clear rule to the remainder.
- **Settlement above the hold:** accept or reject it under an explicit product rule.
- **Refund:** post a new financial entry; do not rewrite the completed authorization.

3. What was cut and why

The implementation is intentionally small. Each cut keeps the core behavior easy to inspect, but it also postpones a production risk.

Simplification	Why it was cut	Production risk deferred
In-memory storage	Keep replay simple	Data is lost if the process stops
Full ledger scans	Data set is small	Calculations slow as history grows
One ordered process	Deterministic replay	Concurrent updates are not protected
No event idempotency	Events are supplied once	A duplicate may post money twice
Fixed rules and six days	Match the scenario	Longer periods and rule changes are unsupported
Limited authorization states	Implement needed behavior	Holds may never expire or cancel
Automatic late value dating	Demonstrate recalculation	No approval or correction workflow
One currency per account	No exchange is needed	No foreign-exchange accounting
No API or access control	Not needed for replay	Cannot safely accept external requests
Minimal monitoring	Local execution is enough	Failures may go undetected or unrecovered

Production priority

1	Durable storage and duplicate-event protection
2	Controlled value-dated adjustments
3	Complete authorization lifecycle
4	Faster account balance projections
5	Access control, monitoring, and recovery

This order protects correctness and recoverability first, then improves lifecycle coverage and speed. It avoids adding distributed infrastructure before the ledger has durable records and safe input handling.

Official sources

[1] [CBUAE Consumer Protection Standards](#) - detailed statements and transaction information.

[2] [CBUAE Operational Risk Standards](#) - approvals, separation of duties, dual control, and reconciliation.